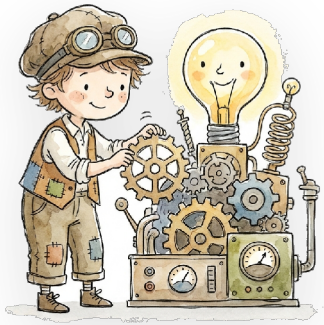
CALCULUS FOR OPTIMIZATION

Differential Calculus: Derivatives and Gradients



YOUR GUIDE

Let's get started.

We can begin with the question that quietly runs every neural network on Earth: **we are standing somewhere on a landscape in the fog, and we want to get downhill. Which way do we step?** This represents the fundamental problem of learning a model; however, the "landscape" is the error of our predictions, the "somewhere" is our current parameters, and "downhill" means less wrong. We cannot see the whole terrain. We can feel the slope right under our feet. Differential calculus is the tool that reads that slope, and the **gradient** is the compass it hands us. By the end

of this chapter, that compass will not be a mystery. Instead, it will be a small pile of arithmetic we can compute on a napkin.

We will build this concept the way it actually earns its keep in machine learning. We compute the derivative first, on one variable, with real numbers; however, we then make the jump to many variables, which is where the word *gradient* shows up and where every model actually lives.

The derivative: how fast is this changing, right here?

We can start with the simplest thing that changes. Let $f(x) = x^2$. For example, this could be a loss function. We can think of x as a single knob on our model and $f(x)$ as how wrong we are when the knob is set to x. We want the x that makes f small. This is $x = 0$ here; however, we will pretend we cannot see the whole curve. We are sitting at $x = 3$, where $f(3) = 9$, and we want to know: **if we nudge x a little, does our error go up or down, and how fast?**

The way to ask is to actually nudge it. Take a tiny step h and measure the change:

$$\frac{f(x+h) - f(x)}{h}$$

This is the **average rate of change** over the little interval from `x` to `x+h`; however, this represents the rise over run, which is the slope of the line connecting two nearby points on the curve. Next, we can observe what happens as we shrink `h` at `x = 3`:

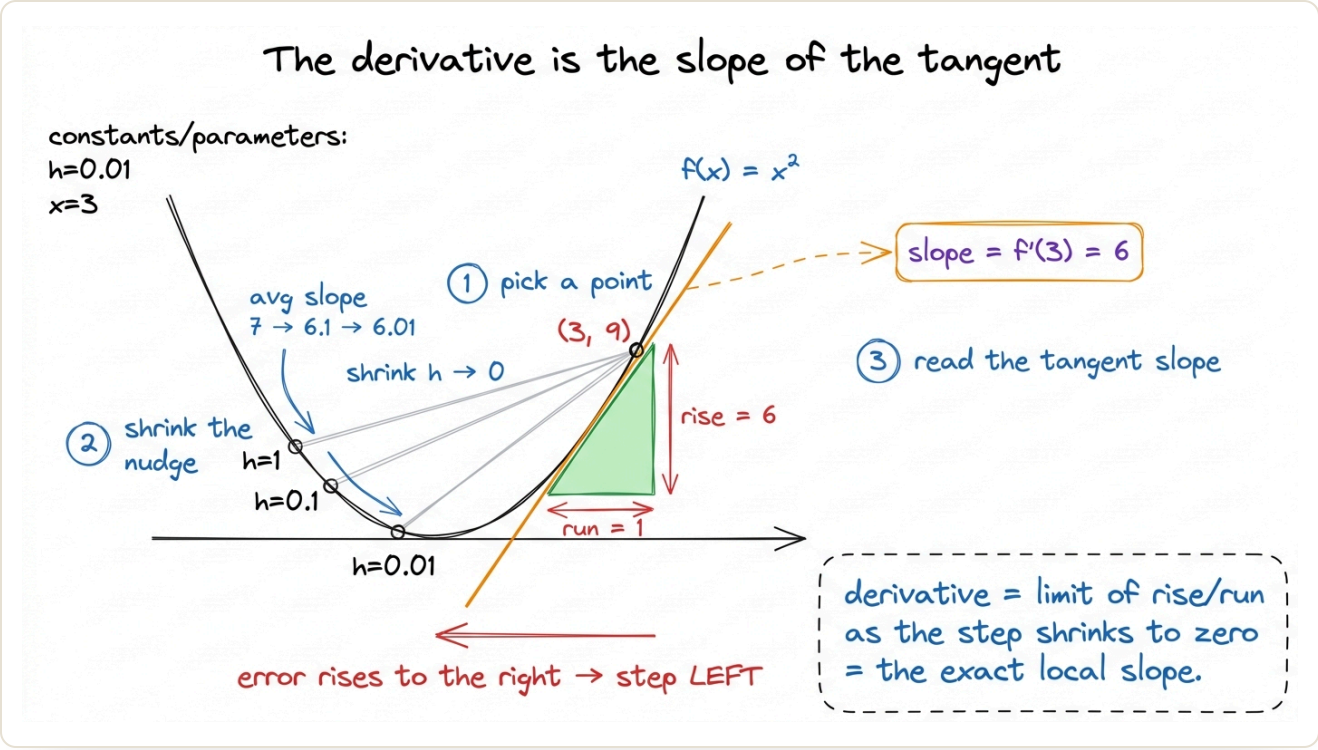
- `h = 1`: `(f(4) - f(3)) / 1 = (16 - 9) / 1 = 7`
- `h = 0.1`: `(f(3.1) - f(3)) / 0.1 = (9.61 - 9) / 0.1 = 6.1`
- `h = 0.01`: `(9.0601 - 9) / 0.01 = 6.01`
- `h = 0.001`: `→ 6.001`

As the output shows, the numbers are moving toward **6**. That limit is the **derivative.** It is written as `f'(x)` or `df/dx`; however, it represents the *instantaneous* rate of change, which is the slope of the tangent line that touches the curve at that one point.[1]

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

For `f(x) = x²` the answer is `f'(x) = 2x`, so `f'(3) = 6`; however, this is exactly what our shrinking `h` was moving toward. The sign is the part that

matters for learning. For example, `f'(3) = +6` is *positive*, which means "at `x = 3`, increasing `x` increases the error." Therefore, to reduce error, we go the *other* way; however, this means we decrease `x`. That single principle, **step against the sign of the derivative**, is gradient descent in one dimension.



As the nudge h shrinks, the secant slope converges to the tangent slope: f'(3) = 6. Its sign tells you which way is downhill.

We almost never compute that limit by hand in practice. A handful of **rules** perform all the work, and these four cover most of ML:

- **Power rule:** `d/dx xⁿ = n·xⁿ⁻¹`. (So `x²` → `2x`, and a constant → `0`.)
- **Sum rule:** the derivative of a sum is the sum of the derivatives.
- **Product rule:** `d/dx (u·v) = u'v + uv'`.
- **Chain rule:** `d/dx f(g(x)) = f'(g(x)) · g'(x)` — the composition rule that the next chapter turns into all of backpropagation.

We can look at one worked line to make the power and sum rules concrete. Let a tiny loss be `L(w) = w² − 4w + 5`. Then, we compute `L'(w) = 2w − 4`. Setting `L'(w) = 0` gives `w = 2`, and that flat spot is the bottom of the bowl; however, this is the **minimum**, where the slope vanishes and there is no more downhill to find.[2] Note that "slope is zero at the optimum" is the mathematical fingerprint of *done*.

From one knob to many: the gradient

A real model does not have one knob. A small neural network has thousands; a large one has billions. As a result, our loss is no longer `f(x)`

but `f(x₁, x₂, …, x_n)`; however, this is a function of many parameters at once. How do we talk about "slope" when there are a thousand directions we could step in?

We can compute this by choosing to **change one variable at a time and hold the rest fixed.** The derivative we obtain this way is called a **partial derivative**, written with a curly `∂` instead of `d`. For example, it answers how fast the loss changes as we adjust *only* `x₁`, assuming that `x₂` is frozen.

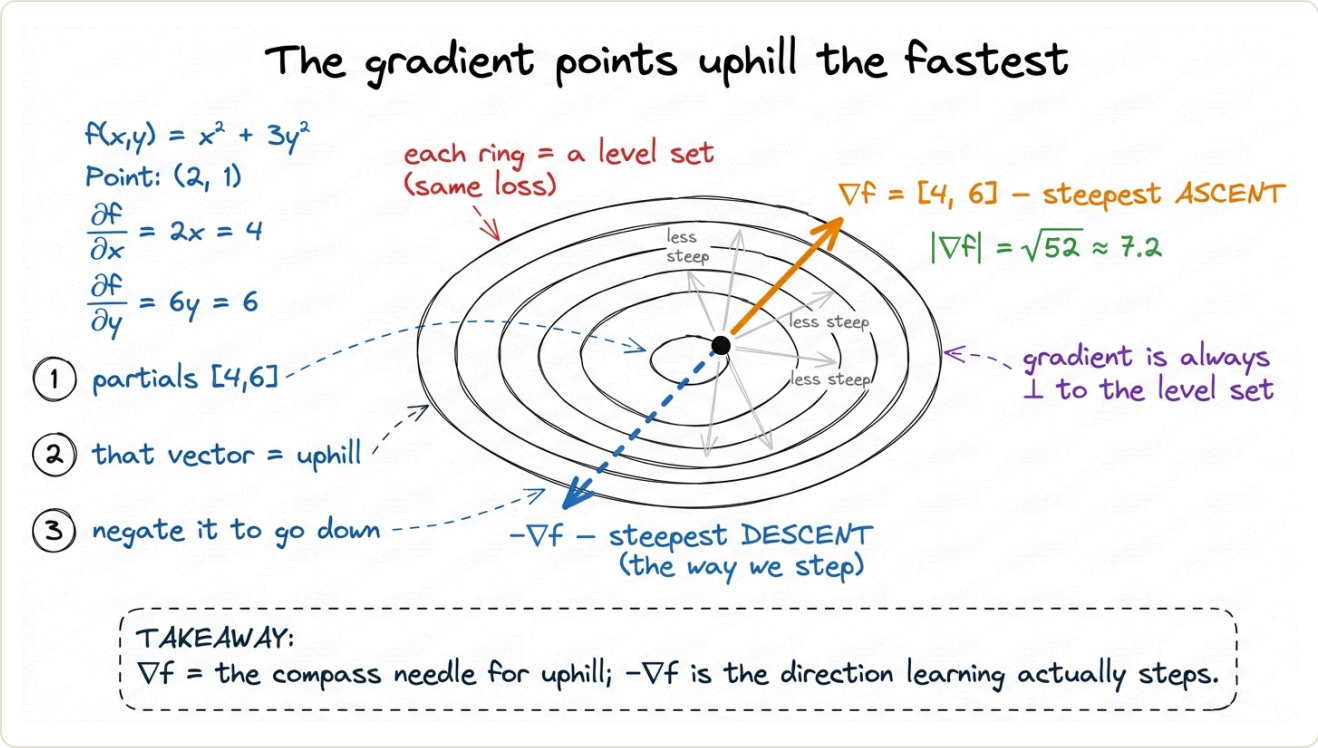
For example, we can consider a concrete two-variable equation: `f(x, y) = x² + 3y²`. We freeze `y` and differentiate in `x`. The `3y²` term is a constant, so it becomes zero, and we compute `∂f/∂x = 2x`. Next, we freeze `x` and differentiate in `y`. The `x²` term becomes zero, and we compute `∂f/∂y = 6y`. We can evaluate both at the point `(x, y) = (2, 1)`:

- `∂f/∂x = 2·2 = 4`
- `∂f/∂y = 6·1 = 6`

We can stack those two numbers into a vector, and we obtain the **gradient**, which is written as `∇f` (pronounced "grad f" or "del f"):

$$\nabla f = \left[\frac{\partial f}{\partial x},\ \frac{\partial f}{\partial y}\right] = [\,4,\ 6\,] \quad \text{at } (2, 1)$$

The gradient is not a slope; however, it is a *direction plus a magnitude*. It has one property that makes the field of optimization possible. Note that **the gradient points in the direction of steepest ascent.** This is the single direction, out of all infinitely many we could step in, along which the function increases fastest. Its length, here $\sqrt{(4^2 + 6^2)} = \sqrt{52} \approx 7.2$, represents *how steep* that steepest climb is.



Each partial derivative is a per-axis slope; stacked, they form the gradient, which points in the steepest-uphill direction and sits perpendicular to the contour lines.

Why does that vector-of-partials happen to point *steepest* uphill? We can find the reason in the dot product we covered in the linear-algebra chapters. If we step a tiny unit vector `u` away from our point, the change in `f` is `∇f · u`. Note that this is the dot product of the gradient with our chosen direction. A dot product `|∇f||u|cos θ` is largest when `θ = 0`, which means when `u` points *the same way as* `∇f`. Therefore, the direction that increases `f` fastest is the gradient's own direction. As a result, linear algebra provides calculus with the mathematical reason this directional method works.

Turning the compass into learning: gradient descent

We can now apply this concept. We wanted to move downhill, and the gradient gives us the uphill direction; however, we can simply walk the *other* way. This is the update rule that trains essentially every model in this book:

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

Here, `θ` is our bundle of parameters, `∇L(θ)` is the gradient of the loss at the current parameters, and `η` (eta) is the **learning rate**. Note that this is a

small positive number that sets how big a step we take.[3] The minus sign indicates that we step *against* steepest ascent, which results in steepest descent.

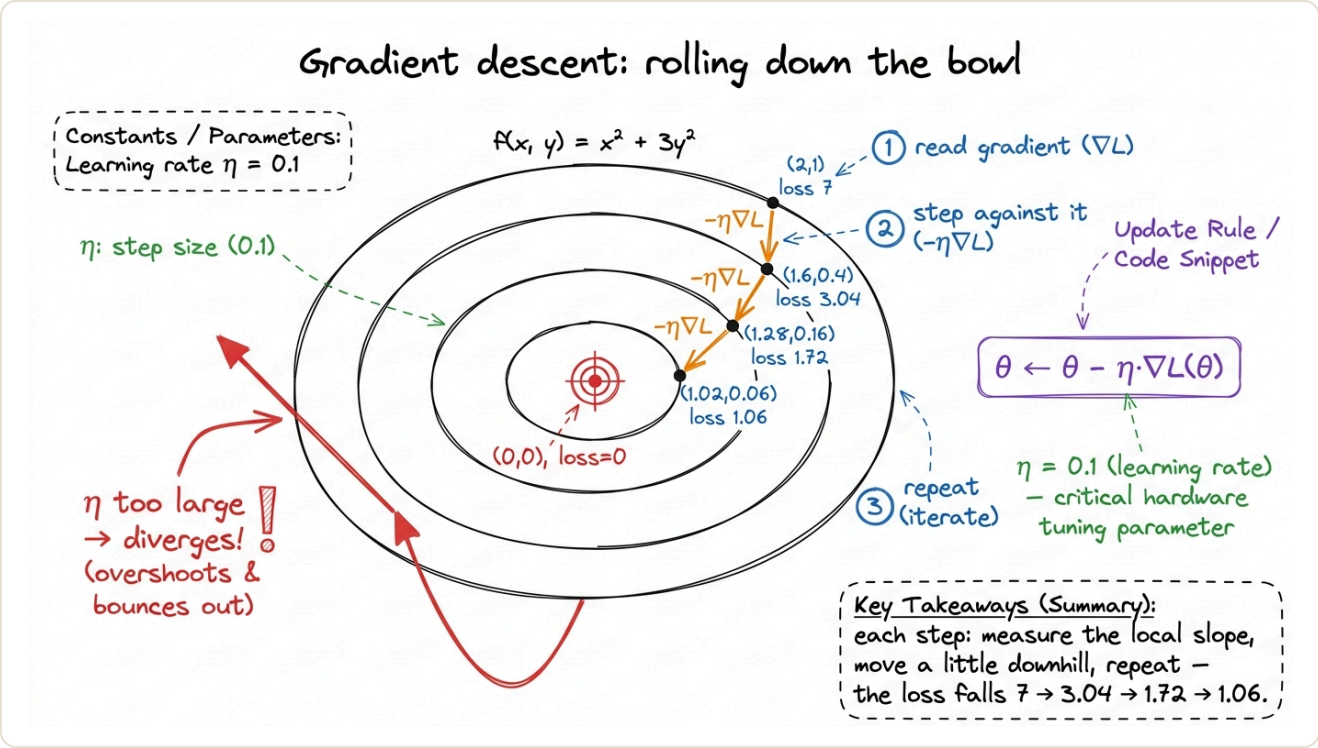
We can compute three steps on `f(x, y) = x² + 3y²` from `(2, 1)` with `η = 0.1`, so we can observe the loss fall manually. As a reminder, the gradient is `∇f = [2x, 6y]`.

- **Start:** `(x, y) = (2, 1)`, loss `= 4 + 3 = 7`. Gradient `= [4, 6]`.
- **Step 1:** `x ← 2 − 0.1·4 = 1.6`; `y ← 1 − 0.1·6 = 0.4`. New loss `= 1.6² + 3·0.4² = 2.56 + 0.48 = 3.04`.
- **Step 2:** gradient at `(1.6, 0.4)` is `[3.2, 2.4]`; `x ← 1.6 − 0.32 = 1.28`; `y ← 0.4 − 0.24 = 0.16`. Loss `= 1.638 + 0.077 = 1.715`.
- **Step 3:** gradient `[2.56, 0.96]`; `x ← 1.024`; `y ← 0.064`. Loss `≈ 1.061`.

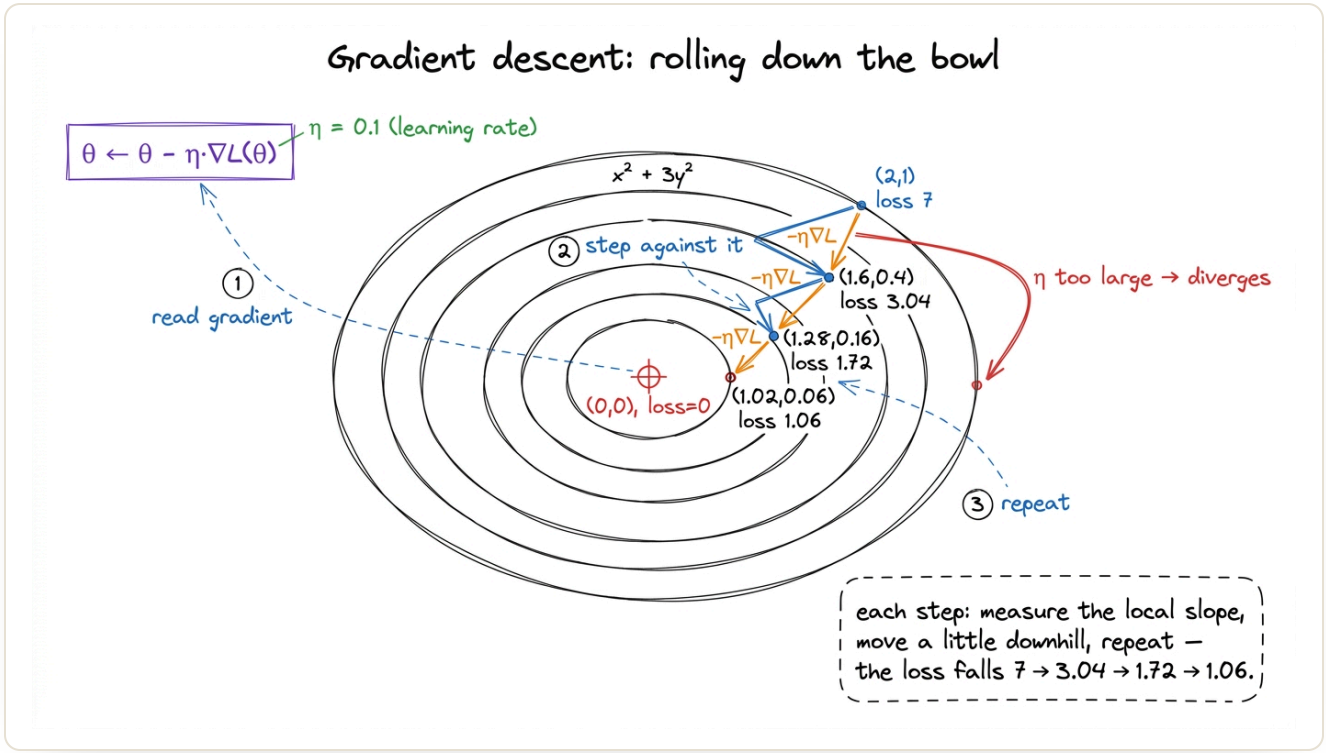
As the output shows, the loss went **7 → 3.04 → 1.72 → 1.06**, moving toward its true minimum of `0` at `(0, 0)`. We achieved this without ever seeing the whole surface; however, we used only the local gradient at each spot. This represents learning, where we repeat the update until the gradient becomes zero.

The pitfall everyone hits first: the step that overshoots

This is a common pitfall, and we can demonstrate it with numbers rather than just reading a warning. Note that the gradient only tells us the *direction* downhill and how steep it is *right here*. It says nothing about how far the valley is; however, a step of size η can overshoot the bottom and land higher up the far wall. For example, we can observe the y coordinate of the same bowl $f(x, y) = x^2 + 3y^2$, whose y-slope is $\partial f/\partial y = 6y$, but now with a large $\eta = 0.4$ starting from $y = 1$. In step one, we compute $y \leftarrow 1 - 0.4 \cdot 6 = 1 - 2.4 = -1.4$. We wanted to move toward 0, but instead we moved past it to -1.4, which is *further* from the minimum than we started. In step two, the gradient is $6 \cdot (-1.4) = -8.4$, so we compute $y \leftarrow -1.4 - 0.4 \cdot (-8.4) = -1.4 + 3.36 = +1.96$. We are now at 1.96, which is further still. The updates are not converging; however, they are *exploding* outward, and the loss climbs $3 \to 5.88 \to 11.5$ instead of falling. **A gradient step points the right way but can still be the wrong size, and too big a learning rate turns descent into divergence.** The solution is not a cleverer direction, since the direction was correct every time. Instead, we use a smaller η: for this bowl, any $\eta < 1/3$ keeps the y-updates shrinking toward 0.



With η too large the update keeps the correct downhill direction yet overshoots the minimum on every step, so the loss diverges instead of falling.



Gradient descent walks against the gradient in small steps, and the loss falls monotonically toward the minimum even though we only ever see the local slope.

We can express this same loop as ten lines of NumPy code. Note that this implementation is exactly what `torch` or `keras` runs under the hood, however, we omit the additional framework machinery.

```
import numpy as np

def loss(p):          # p = [x, y]
    return p[0]**2 + 3*p[1]**2
```

```
def grad(p):          # ∇f = [∂f/∂x, ∂f/∂y]
    return np.array([2*p[0], 6*p[1]])

p = np.array([2.0, 1.0])   # starting parameters
eta = 0.1                  # learning rate
for step in range(4):
    print(step, p, loss(p))
    p = p - eta * grad(p)  # θ ← θ − η ∇L(θ)
```

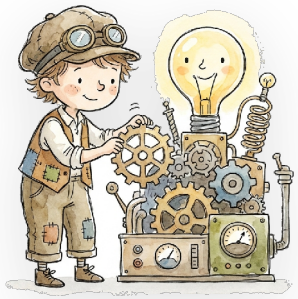
Note that there is no calculus *at runtime*, however, we worked out `grad` once by hand, and the loop simply evaluates and subtracts. In a real network, we have thousands of parameters and we cannot differentiate them by hand. Therefore, the framework's **autograd** builds the gradient for us by applying the chain rule across the whole computation. This concept provides the bridge to what comes next.

The one thing to carry forward

We can summarize the primary concept from this chapter as follows: **the derivative is a sign that tells us which way is downhill, and the gradient applies that same idea in every direction at once, representing a vector that points steepest-uphill, however, we step against it.** Every optimizer

in this book, such as plain gradient descent, SGD, momentum, and Adam, is a variation on that single procedure. For example, we read `∇L`, take a step of size `η` in the `−∇L` direction, and repeat. As we observed earlier, the loss obeys this process, falling `7 → 1.06` in three napkin steps.

We computed today's gradients by hand because the functions were tiny. A neural network is a deep *composition* of functions, meaning we have layer inside layer inside layer, however, we do not differentiate those by hand. The rule that lets a machine perform this computation automatically, propagating slopes backward from the loss to every weight, is the chain rule. This concept is exactly where we go next in the chain rule and backpropagation.



CHAPTER COMPLETE

Nicely done. You reached the end of this one.

NEXT CHAPTER The Chain Rule and Backpropagation →

0
Did this chapter land for you?

Mark chapter as done

Rate this chapter ★ ★ ★ ★ ★

What did you think of this chapter?

A takeaway, a question, a moment that clicked...

Be kind and specific. 0/2000 Post

No notes yet — be the first to share what you took away.

‹ Evaluating a Model: Acc…

Back to book

The Chain Rule and Back… ›